

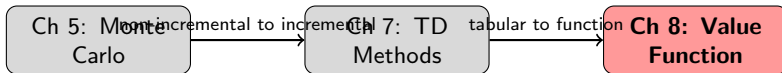
Chapter 8: Value Function Methods

Adapted from Shiyu Zhao's Textbook

Outline

- 1 Introduction
- 2 Value Representation: From Table to Function
- 3 TD Learning with Function Approximation
- 4 TD Learning of Action Values
- 5 Deep Q-Learning (DQN)
- 6 Summary

Where Value Function Methods Fit



Key Transition: From **tabular representation** to **function representation**

- Tabular method: straightforward but inefficient for large state/action spaces
- Function approximation: efficient, scalable, enables neural networks
- This is where **deep learning** meets **reinforcement learning**

- ➊ **Section 8.1:** Value Representation — From Table to Function
 - Basic idea of function approximation
 - Linear vs nonlinear approximators
- ➋ **Section 8.2:** TD Learning of State Values with Function Approximation
 - Objective function and optimization
 - Feature vector selection
 - Theoretical analysis (projected Bellman error)
- ➌ **Section 8.3:** TD Learning of Action Values
 - Sarsa with function approximation
 - Q-learning with function approximation
- ➍ **Section 8.4:** Deep Q-Learning (DQN)
 - Main network and target network
 - Experience replay

Tabular Method Review

Setup: n states $\{s_i\}_{i=1}^n$ with state values $\{v_\pi(s_i)\}_{i=1}^n$

Tabular representation:

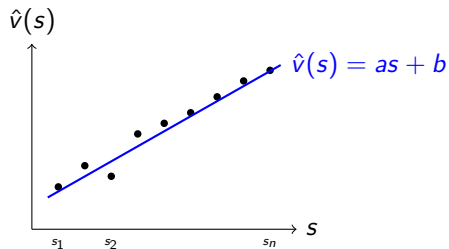
State	s_1	s_2	$\dots$	s_n
Estimated value	$\hat{v}(s_1)$	$\hat{v}(s_2)$	$\dots$	$\hat{v}(s_n)$

Properties:

- Stored as an array or vector in memory
- **Retrieve:** directly read corresponding entry
- **Update:** directly rewrite corresponding entry
- **Problem:** requires $O(n)$ storage, inefficient for large n

Function Approximation: Basic Idea

Key Insight: Fit the points $\{(s_i, \hat{v}(s_i))\}_{i=1}^n$ with a curve/function

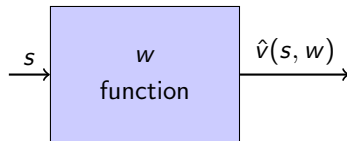


Simplest approximation: straight line

$$\hat{v}(s, w) = as + b = \underbrace{[s, 1]}_{\phi^T(s)} \underbrace{\begin{bmatrix} a \\ b \end{bmatrix}}_w = \phi^T(s)w$$

Notation: $\hat{v}(s, w)$: approximated value, $w \in \mathbb{R}^2$: parameter vector, $\phi(s) \in \mathbb{R}^2$: feature vector

How to Retrieve a Value



Process:

- Input state s into function
- Calculate feature vector $\phi(s)$
- Compute $\hat{v}(s, w) = \phi^T(s)w$
- For neural networks: forward propagation from input to output

Storage efficiency: Only need to store parameter vector w (dimension $m \ll n$)

How to Update a Value

Key difference from tabular method:

- **Cannot** directly change $\hat{v}(s)$
- Must update parameter w to change values indirectly
- Updating w affects all state values!

This leads to generalization:

- With tabular method: updating $\hat{v}(s_3)$ does not change $\hat{v}(s_1)$ or $\hat{v}(s_2)$
- With function approximation: updating w for s_3 also changes values of s_1, s_2
- Experience from one state helps estimate values of neighboring states
- This is a key advantage over the tabular method

Higher-Order Polynomial Approximations

Second-order polynomial:

$$\hat{v}(s, w) = as^2 + bs + c = \underbrace{[s^2, s, 1]}_{\phi^T(s)} \underbrace{\begin{bmatrix} a \\ b \\ c \end{bmatrix}}_w = \phi^T(s)w$$

Trade-off:

- Higher order $\Rightarrow$ better approximation accuracy
- Higher order $\Rightarrow$ larger w , more storage/computation

Linear Function Approximation

Note: $\hat{v}(s, w) = \phi^T(s)w$ is **linear in w** (though possibly nonlinear in s).
This is called **linear function approximation**.

Linear vs Nonlinear Function Approximation

Linear Function Approximation:

$$\hat{v}(s, w) = \phi^T(s)w$$

- Linear in parameter w
- Requires selecting feature vector $\phi(s)$
- Well-understood theory
- Limited approximation ability

Nonlinear (Neural Network):

$$\hat{v}(s, w) = \text{NN}(s; w)$$

- Universal approximator
- No manual feature engineering
- More complex theory
- Powerful but may overfit

Optimal Parameter via Least Squares

If we knew the true values $\{v_\pi(s_i)\}_{i=1}^n$, finding optimal w is a least-squares problem:

$$J_1 = \sum_{i=1}^n (\hat{v}(s_i, w) - v_\pi(s_i))^2 = \sum_{i=1}^n (\phi^T(s_i)w - v_\pi(s_i))^2$$

In matrix form:

$$J_1 = \|\Phi w - v_\pi\|^2, \quad \text{where } \Phi = \begin{bmatrix} \phi^T(s_1) \\ \vdots \\ \phi^T(s_n) \end{bmatrix} \in \mathbb{R}^{n \times m}$$

Optimal solution:

$$w^* = (\Phi^T \Phi)^{-1} \Phi^T v_\pi$$

Problem: We don't know v_π — it must be estimated!

Objective Function

Goal: Find optimal w so that $\hat{v}(s, w)$ best approximates $v_\pi(s)$ for all s .

Objective Function

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S, w))^2]$$

Key question: What is the probability distribution of S ?

Two options:

- ① **Uniform distribution:** All states equally important

$$J(w) = \frac{1}{n} \sum_{s \in \mathcal{S}} (v_\pi(s) - \hat{v}(s, w))^2$$

- ② **Stationary distribution $d_\pi(s)$:** Weight by long-term visit frequency

$$J(w) = \sum_{s \in \mathcal{S}} d_\pi(s) (v_\pi(s) - \hat{v}(s, w))^2$$

Gradient Descent for Optimization

Gradient descent algorithm:

$$w_{k+1} = w_k - \alpha_k \nabla_w J(w_k)$$

Calculate gradient:

$$\begin{aligned}\nabla_w J(w_k) &= \nabla_w \mathbb{E}[(v_\pi(S) - \hat{v}(S, w_k))^2] \\ &= -2\mathbb{E}[(v_\pi(S) - \hat{v}(S, w_k))\nabla_w \hat{v}(S, w_k)]\end{aligned}$$

Gradient descent update:

$$w_{k+1} = w_k + \alpha_k \mathbb{E}[(v_\pi(S) - \hat{v}(S, w_k))\nabla_w \hat{v}(S, w_k)]$$

Stochastic gradient descent (SGD): Replace expectation with sample

$$w_{t+1} = w_t + \alpha_t (v_\pi(s_t) - \hat{v}(s_t, w_t))\nabla_w \hat{v}(s_t, w_t)$$

Problem: Still requires unknown true value $v_\pi(s_t)$!

Approximating the True Value

Problem: $v_\pi(s_t)$ is unknown and must be approximated.

Two approaches:

1. Monte Carlo Method

Use discounted return g_t as approximation of $v_\pi(s_t)$:

$$w_{t+1} = w_t + \alpha_t (g_t - \hat{v}(s_t, w_t)) \nabla_w \hat{v}(s_t, w_t)$$

2. Temporal-Difference Method

Use TD target $r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t)$ as approximation:

$$w_{t+1} = w_t + \alpha_t [r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)] \nabla_w \hat{v}(s_t, w_t)$$

The TD method is the focus of this chapter.

TD Learning Algorithm with Function Approximation

TD Learning Update Rule

$$w_{t+1} = w_t + \alpha_t \underbrace{[r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)]}_{\text{TD error } \delta_t} \nabla_w \hat{v}(s_t, w_t)$$

Linear case: $\hat{v}(s, w) = \phi^T(s)w$, so $\nabla_w \hat{v}(s, w) = \phi(s)$

TD-Linear Algorithm

$$w_{t+1} = w_t + \alpha_t [r_{t+1} + \gamma \phi^T(s_{t+1})w_t - \phi^T(s_t)w_t] \phi(s_t)$$

Algorithm structure:

- Input: experience samples (s_t, r_{t+1}, s_{t+1}) generated by policy π
- Output: parameter w such that $\hat{v}(s, w) \approx v_\pi(s)$

Tabular TD is a Special Case of TD-Linear

Claim: Tabular TD (Chapter 7) is a special case of TD-Linear.

Proof: Choose the feature vector as $\phi(s) = e_s \in \mathbb{R}^n$ (one-hot vector).

Then: $\hat{v}(s, w) = e_s^T w = w(s)$

The TD-Linear update becomes:

$$w_{t+1} = w_t + \alpha_t [r_{t+1} + \gamma w_t(s_{t+1}) - w_t(s_t)] e_{s_t}$$

This only updates entry $w_t(s_t)$:

$$w_{t+1}(s_t) = w_t(s_t) + \alpha_t [r_{t+1} + \gamma w_t(s_{t+1}) - w_t(s_t)]$$

This is exactly the **tabular TD algorithm** from Chapter 7!

Feature Vector Selection: Polynomial Features

Grid world example: State s corresponds to 2D location (x, y)

First-order (3D feature vector):

$$\phi(s) = \begin{bmatrix} 1 \\ x \\ y \end{bmatrix} \in \mathbb{R}^3 \quad \Rightarrow \quad \hat{v}(s, w) = w_1 + w_2x + w_3y$$

Represents a 2D plane.

Second-order (6D feature vector):

$$\phi(s) = [1, x, y, x^2, y^2, xy]^T \in \mathbb{R}^6$$

Represents a quadratic surface.

Third-order (10D feature vector):

$$\phi(s) = [1, x, y, x^2, y^2, xy, x^3, y^3, x^2y, xy^2]^T \in \mathbb{R}^{10}$$

Trade-off: Higher dimension $\Rightarrow$ better approximation but more parameters.

Feature Vector Selection: Fourier Features

Fourier basis features: Normalize $x, y \in [0, 1]$

$$\phi(s) = \begin{bmatrix} \vdots \\ \cos(\pi(c_1x + c_2y)) \\ \vdots \end{bmatrix} \in \mathbb{R}^{(q+1)^2}$$

where $c_1, c_2 \in \{0, 1, \dots, q\}$.

Example: $q = 1$ gives dimension $(1 + 1)^2 = 4$:

$$\phi(s) = \begin{bmatrix} \cos(\pi(0 \cdot x + 0 \cdot y)) \\ \cos(\pi(0 \cdot x + 1 \cdot y)) \\ \cos(\pi(1 \cdot x + 0 \cdot y)) \\ \cos(\pi(1 \cdot x + 1 \cdot y)) \end{bmatrix} = \begin{bmatrix} 1 \\ \cos(\pi y) \\ \cos(\pi x) \\ \cos(\pi(x + y)) \end{bmatrix}$$

Dimensions: $q = 1 \Rightarrow 4$, $q = 2 \Rightarrow 9$, $q = 3 \Rightarrow 16$

Experimental Results: Effect of Feature Dimension

Setup: Estimate state values of a uniform random policy in grid world

Feature Type	Dimension	Final Error
Polynomial $[1, x, y]^T$	3	High (2D plane)
Polynomial $[1, x, y, x^2, y^2, xy]^T$	6	Medium
Polynomial (3rd order)	10	Lower
Fourier $q = 1$	4	High
Fourier $q = 2$	9	Medium
Fourier $q = 3$	16	Lower

Key observations:

- Higher feature dimension $\Rightarrow$ more accurate approximation
- Error cannot converge to zero with linear approximation (limited capacity)
- Trade-off between accuracy and computational cost

Sarsa with Function Approximation

Goal: Estimate action values $q_\pi(s, a)$ using function $\hat{q}(s, a, w)$

Replace state values with action values in TD update:

Sarsa with Function Approximation

$$w_{t+1} = w_t + \alpha_t [r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t)] \nabla_w \hat{q}(s_t, a_t, w_t)$$

Linear case: $\hat{q}(s, a, w) = \phi^T(s, a)w$

- Feature vector $\phi(s, a)$ depends on both state and action
- Gradient: $\nabla_w \hat{q}(s, a, w) = \phi(s, a)$

Combine with policy improvement:

- Use ϵ -greedy policy based on current $\hat{q}$
- Update policy after each value update

Algorithm: Sarsa with Function Approximation

Sarsa with Function Approximation

Initialize: parameter w_0 , policy π_0 , learning rate α , exploration ϵ

For each episode:

- ① Generate a_0 at s_0 following $\pi_0(s_0)$
- ② **While** s_t is not terminal:
 - Collect $(r_{t+1}, s_{t+1}, a_{t+1})$
 - Update: $w_{t+1} = w_t + \alpha[r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t)] \nabla_w \hat{q}(s_t, a_t, w_t)$
 - Update policy to ϵ -greedy based on $\hat{q}(\cdot, \cdot, w_{t+1})$
 - $s_t \leftarrow s_{t+1}, a_t \leftarrow a_{t+1}$

Note: Policy $\pi(a|s)$ is still represented as a table. Chapter 9 will use policy functions.

Q-Learning with Function Approximation

Replace $\hat{q}(s_{t+1}, a_{t+1}, w_t)$ with $\max_a \hat{q}(s_{t+1}, a, w_t)$:

Q-Learning with Function Approximation

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \max_{a \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a, w_t) - \hat{q}(s_t, a_t, w_t) \right] \nabla_w \hat{q}(s_t, a_t, w_t)$$

Comparison of TD targets:

Algorithm	TD Target
Sarsa	$r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t)$
Q-learning	$r_{t+1} + \gamma \max_a \hat{q}(s_{t+1}, a, w_t)$

Properties:

- Can be implemented on-policy or off-policy
- Off-policy: learn from experience generated by any behavior policy

Deep Q-Learning: Introduction

Deep Q-Learning / Deep Q-Network (DQN)

Integrate **deep neural networks** into Q-learning as nonlinear function approximators.

Historical significance:

- One of the earliest and most successful deep RL algorithms
- Achieved human-level performance on Atari games (Mnih et al., 2015)
- Sparked the deep reinforcement learning revolution

Key differences from standard Q-learning:

- ① Uses neural network instead of linear function
- ② Requires special techniques for stable training:
 - Experience replay
 - Target network

DQN Objective Function

Objective: Minimize the squared Bellman optimality error

DQN Objective Function

$$J = \mathbb{E} \left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w) \right)^2 \right]$$

Intuition: The Bellman optimality equation is

$$q(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \max_a q(S_{t+1}, a) \mid S_t = s, A_t = a \right]$$

So the term inside the square should be zero when $\hat{q}$ perfectly approximates q^* .

Problem: w appears in both $\hat{q}(S, A, w)$ and the target $R + \gamma \max_a \hat{q}(S', a, w)$.
Computing the gradient is complicated!

Target Network

Solution: Use two networks with different parameters

Main Network: $\hat{q}(s, a, w)$

- Parameter: w
- Updated every iteration
- Used to select actions

Target Network: $\hat{q}(s, a, w_T)$

- Parameter: w_T
- Fixed for C iterations
- Used to compute TD target

Modified objective:

$$J = \mathbb{E} \left[\left(\underbrace{R + \gamma \max_a \hat{q}(S', a, w_T)}_{\text{TD target (fixed)}} - \hat{q}(S, A, w) \right)^2 \right]$$

Gradient with Target Network

Gradient (with w_T fixed):

$$\nabla_w J = -\mathbb{E} \left[(R + \gamma \max_a \hat{q}(S', a, w_T) - \hat{q}(S, A, w)) \nabla_w \hat{q}(S, A, w) \right]$$

Why two networks?

- Simplifies gradient computation
- Target network provides stable learning targets
- Without it: “chasing a moving target” instability
- Update target network periodically (every C steps): $w_T \leftarrow w$

Experience Replay

Problem: Samples $(s_t, a_t, r_{t+1}, s_{t+1})$ are correlated (sequential)

Solution: Experience Replay

Experience Replay Buffer

- 1 Store experience samples in a **replay buffer**: $\mathcal{B} = \{(s, a, r, s')\}$
- 2 When training, **uniformly sample** a mini-batch from $\mathcal{B}$
- 3 Use mini-batch to update the main network

Why is experience replay necessary?

- Objective function assumes (S, A) is uniformly distributed
- Sequential samples violate this assumption (correlated)
- Random sampling breaks correlation
- Each sample can be reused multiple times (data efficiency)

DQN Algorithm

Deep Q-Learning (Off-Policy Version)

Initialize: Main network $\hat{q}(s, a, w)$, target network $\hat{q}(s, a, w_T)$ with $w_T = w$

Collect: Store experience samples from behavior policy π_b in replay buffer $\mathcal{B}$

For each iteration:

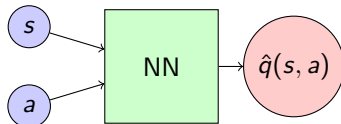
- 1 Uniformly sample mini-batch $\{(s, a, r, s')\}$ from $\mathcal{B}$
- 2 For each sample, compute TD target: $y_T = r + \gamma \max_{a' \in \mathcal{A}(s')} \hat{q}(s', a', w_T)$
- 3 Update main network to minimize loss: $\text{Loss} = \sum_{\text{batch}} (y_T - \hat{q}(s, a, w))^2$
- 4 Every C iterations: $w_T \leftarrow w$

Key: Use standard neural network training tools (backpropagation, Adam, etc.)

DQN: Network Architecture

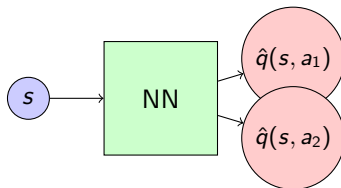
Two common designs:

Design 1: State-Action Input



Input: (s, a) , Output: single value $\hat{q}(s, a)$

Design 2: State Input Only



Input: s only, Output: all action values (more efficient for $\max_a$)

DQN Example: Grid World

Setup:

- Behavior policy: uniform (highly exploratory)
- Episode: 1,000 steps
- Network: 1 hidden layer with 100 neurons
- Input: normalized (row, column, action)
- Mini-batch size: 100

Results:

- Loss function converges to zero
- State value error converges to zero
- Optimal policy is learned

Efficiency

DQN with 1,000 steps achieves what tabular Q-learning needs 100,000 steps for!

Reasons: (1) Generalization ability (2) Sample reuse via replay

DQN: Effect of Data Quantity

Experiment: Compare 1,000 steps vs 100 steps

With 1,000 steps:

- Loss converges to 0 (good)
- Value error converges to 0 (good)
- Optimal policy learned (good)

With 100 steps:

- Loss converges to 0 (good)
- Value error does NOT converge to 0 (bad)
- Suboptimal policy

Key insight: Low loss does not mean accurate value estimates. Network fits training data well, but data is insufficient. This is **overfitting** to limited experience.

Chapter Summary

Key transition: Tabular representation $\rightarrow$ Function approximation

Main concepts:

- ① **Function approximation:** $\hat{v}(s, w) = \phi^T(s)w$ or neural network
 - More storage efficient
 - Better generalization
 - Tabular is a special case
- ② **TD with function approximation:**
 - Minimizes projected Bellman error (not true error)
 - Convergence guaranteed for linear case
- ③ **Deep Q-Learning (DQN):**
 - Neural network as function approximator
 - Two key techniques: target network + experience replay
 - Much more sample efficient than tabular methods

Comparison: Tabular vs Function Approximation

	Tabular	Function Approx.
Storage	$O(S)$	$O(m)$ where $m \ll S $
Retrieve value	Direct lookup	Compute $\phi^T(s)w$
Update value	Change single entry	Update w (affects all)
Generalization	None	Strong
Scalability	Limited	Scalable
Theory	Well understood	More complex

When to use which:

- Small discrete state space $\Rightarrow$ Tabular (simpler, exact)
- Large/continuous state space $\Rightarrow$ Function approximation (scalable)

Key Equations Summary

TD Learning with Function Approximation:

$$w_{t+1} = w_t + \alpha_t [r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)] \nabla_w \hat{v}(s_t, w_t)$$

Sarsa with Function Approximation:

$$w_{t+1} = w_t + \alpha_t [r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t)] \nabla_w \hat{q}(s_t, a_t, w_t)$$

Q-Learning with Function Approximation:

$$w_{t+1} = w_t + \alpha_t [r_{t+1} + \gamma \max_a \hat{q}(s_{t+1}, a, w_t) - \hat{q}(s_t, a_t, w_t)] \nabla_w \hat{q}(s_t, a_t, w_t)$$

DQN Objective:

$$J = \mathbb{E} \left[\left(R + \gamma \max_a \hat{q}(S', a, w_T) - \hat{q}(S, A, w) \right)^2 \right]$$

Looking Ahead

What we have covered:

- Value function approximation
- TD, Sarsa, Q-learning with function approximation
- Deep Q-learning (DQN)

What is next (Chapter 9: Policy Gradient Methods):

- So far: values represented as functions, policies as tables
- Next: **policies** represented as functions too!
- Direct optimization of policy parameters
- Handles continuous action spaces naturally

Chapter 10: Actor-Critic Methods

- Combine value-based and policy-based methods
- Best of both worlds